

Introduction to MongoDB — Hands-On Lab

MERN Stack Bootcamp · Week 5 · Day 22 · Companion to Lecture Day 22

Duration: ~30 minutes | **Difficulty:** Beginner | **Tools:** MongoDB Atlas (or local) + Compass or mongosh

Lab overview

In this lab you will connect to MongoDB, create a **shop_db** database, seed a **products** collection, and practice **CRUD** operations and query operators covered in today's lecture. You will finish by creating an index and reading an **explain()** plan — the same skills you will use when wiring Express + Mongoose in upcoming classes.

What you will practice

- Connect with a connection string (Atlas or localhost)
- insertOne / insertMany — create documents
- find / findOne — read with filters and projection
- updateOne — \$set and \$inc operators
- deleteOne — remove a document safely
- Query operators: \$gt, \$in, \$or
- createIndex + explain("executionStats")

Section	Task	Time
0	Connect to MongoDB (Atlas or local)	5 min
1	Create database & seed products	8 min
2	Read & query with operators	8 min
3	Update & delete documents	5 min
4	Create index & run explain()	4 min
	Total	~30 min

Phase 0 Connect to MongoDB

5 min

Option A — MongoDB Atlas (recommended)

- Sign in at <https://cloud.mongodb.com> and open your cluster.
- Click **Connect** → **Compass** or **Shell** and copy the connection string.
- Replace **<password>** with your database user password.
- In Compass: paste the string and connect. In mongosh: run `mongosh "your-uri"`

Option B — Local MongoDB

```
mongosh "mongodb://localhost:27017"
```

Security note: Never commit your connection string or password to Git. Use `.env` when you connect from Node later.

Checkpoint: You see a prompt like `Atlas atlas-xxxxx-shard-0 [primary] test>` or `test>` — you are connected.

Phase 1 Create database & seed products

8 min

MongoDB creates a database and collection when you first insert data. Switch to `shop_db` and insert five product documents.

Step 1.1 — Switch database

```
use shop_db
```

Step 1.2 — Insert sample products

```
db.products.insertMany([
  { title: "Wireless Mouse", price: 29.99, category: "accessories", inStock: true, tags: [
    "wireless", "ergonomic" ] },
  { title: "Mechanical Keyboard", price: 89.99, category: "accessories", inStock: true,
    tags: ["mechanical", "rgb" ] },
  { title: "USB-C Hub", price: 45.00, category: "accessories", inStock: false, tags: ["usb-
    c" ] },
  { title: "27-inch Monitor", price: 299.99, category: "displays", inStock: true, tags: [
    "4k", "ips" ] },
  { title: "Laptop Stand", price: 39.99, category: "accessories", inStock: true, tags: [
    "ergonomic" ] }
])
```

Step 1.3 — Verify the insert

```
db.products.countDocuments()
db.products.find().pretty()
```

Checkpoint: `countDocuments()` returns **5**. Each document has an auto-generated `_id` field.

Phase 2 Read & query with operators

8 min

Step 2.1 — Find all in-stock accessories

```
db.products.find({ category: "accessories", inStock: true })
```

Step 2.2 — Price greater than \$40 (\$gt)

```
db.products.find({ price: { $gt: 40 } })
```

Step 2.3 — Match categories with \$in

```
db.products.find({ category: { $in: ["accessories", "displays"] } })
```

Step 2.4 — \$or — sale candidates (low price OR out of stock)

```
db.products.find({
  $or: [
```

```

    { price: { $lt: 40 } },
    { inStock: false }
  ]
})

```

Step 2.5 — Projection (only title and price)

```
db.products.find({}, { title: 1, price: 1, _id: 0 })
```

Step 2.6 — findOne by title

```
db.products.findOne({ title: "27-inch Monitor" })
```

Checkpoint: \$gt query returns 3 documents. \$or returns at least USB-C Hub and Laptop Stand. Projection output shows only title and price.

Phase 3 Update & delete documents

5 min

Step 3.1 — Update price with \$set

```

db.products.updateOne(
  { title: "Wireless Mouse" },
  { $set: { price: 24.99, onSale: true } }
)

```

Step 3.2 — Increment a numeric field with \$inc

```

// Add views field if missing; increment by 1
db.products.updateOne(
  { title: "Wireless Mouse" },
  { $inc: { views: 1 } }
)

```

Step 3.3 — Verify the update

```
db.products.findOne({ title: "Wireless Mouse" })
```

Step 3.4 — Delete one document

Warning: deleteMany({}) removes every document in the collection. Always use a specific filter.

```

db.products.deleteOne({ title: "USB-C Hub" })
db.products.countDocuments() // should be 4

```

Checkpoint: Mouse document shows price: 24.99, onSale: true, and views: 1. Count is 4 after delete.

Phase 4 Index & explain plan

4 min

Indexes speed up queries on large collections. Create a compound index on category + price, then inspect the execution plan.

Step 4.1 — Create compound index

```
db.products.createIndex({ category: 1, price: -1 })
```

Step 4.2 — List indexes

```
db.products.getIndexes()
```

Step 4.3 — Run explain on a filtered query

```
db.products.find({ category: "accessories" }).explain("executionStats")
```

In the output, look for `executionStats.totalDocsExamined` and whether `winningPlan.inputStage.stage` mentions IXSCAN (index scan) vs COLLSCAN (full collection scan).

Checkpoint: You see at least two indexes: default `_id_` and your `category_1_price_-1` index.

Bonus challenges (if you finish early)

- Insert a user's collection with 3 documents (name, age, email). Query users with age > 25.
- Use \$push to add a tag to one product's tags array.
- Run a simple aggregation: average price per category (see lecture slide).

Lab submission checklist

- Screenshot or copy of `db.products.countDocuments()` showing 4 documents.
- Copy of one `find()` using `$gt` or `$in` and its result count.
- Copy of `getIndexes()` output showing your compound index.
- One sentence: What is the difference between `find()` and `findOne()`?

Troubleshooting

Problem	Fix
Authentication failed	Check username/password in URI; special chars in password must be URL-encoded.
IP not whitelisted (Atlas)	Network Access → Add IP Address → Allow from anywhere (class only) or your IP.
Cannot connect locally	Ensure MongoDB service is running; default port is 27017.
Update did not change doc	Use update operators: <code>{ \$set: { ... } }</code> , not bare fields.
deleteMany deleted everything	Always double-check filter before delete; restore from seed data if needed.

Key takeaways

- Data lives in **documents** inside **collections** inside a **database**.
- Always use update operators (`$set`, `$inc`, ...) in `updateOne`.
- Filters use the same logic you know from JavaScript — comparison and logical operators.
- Indexes matter for performance; `explain()` shows how MongoDB runs your query.

Next class preview: connect MongoDB to Express with Mongoose and replace DummyJSON with your own API.